



**Al-imam Mohammed bin Saud Islamic University**

**College of Computer and Information Sciences**

**Computer Science Department**

**2025 /1446**

**CS330 – Computer Networks**

**Network Programming Project**

**Instructor: Nasser Mohammed Ahmed**

| <b>Student name</b>        | <b>Student ID</b> |
|----------------------------|-------------------|
| <b>Rashid Al-Qahtani</b>   | <b>444001334</b>  |
| <b>Abdullah Alkhawlani</b> | <b>444000843</b>  |
| <b>Mohammed Al-Fayez</b>   | <b>444001229</b>  |

# CS330 Network Programming Project Report

## 1. Introduction

Socket programming is a technique that allows communication between two devices over a network using sockets. A socket is an endpoint for sending or receiving data across a network. This project implements a TCP-based file transfer system using socket programming. The client requests a file, and the server sends the file in fragments, ensuring data integrity using a checksum.

## 2. How TCP File Transfer Works

- The server listens to incoming connections from clients.
- The client initiates a connection using the server's IP and port.
- The client requests a file by sending its filename.
- The server checks if the file exists:
  - If found, the file is fragmented and sent with a checksum.
  - If not found, the client receives an error message.
- The client reassembles the file after verifying the checksum.
- If corruption is detected, retransmissions occur (max 5 retries).

## 3. Implementation Details

This project includes two main components:

- **TCP Server:** Accepts file requests, fragments the file, computes checksums, and transmits data.
- **TCP Client:** Requests the file, verifies checksum, and reconstructs the file.

## 4. Step-by-Step Explanation of Code

### 1: the Client Explanation of Code

[link](#)

#### 1. Connecting to the Server:

- The client connects to the server using an IP and port.
- It sends the requested filename to the server.

#### 2. Receiving File Segments:

- The client receives file segments one by one.
- It extracts the **sequence number, checksum, and segment size**.

#### 3. Checksum Verification:

- The received checksum is compared with the newly calculated checksum.
- If they match, the client sends an **ACK**.
- If they don't match, the client sends a **NACK**, requesting retransmission.

#### 4. Reassembling the File:

- All correctly received segments are stored in a dictionary.
- Once all segments arrive, they are reassembled in order.
- The reconstructed file is saved as received\_<filename>.

## 5. Handling Errors:

- If the received file is incomplete or corrupted, an error message is displayed.
- If the server is unavailable, the client exits gracefully.

## 2: Explanation of the Server Code:

[link](#)

### 1. Initializing the Server:

- The server binds to a specified IP and port (0.0.0.0:8473).
- It starts listening for incoming client connections.

### 2. Handling Client Connections:

- The server accepts client requests and starts a separate thread to handle each connection.
- This allows multiple clients to communicate with the server simultaneously.

### 3. Processing File Requests:

- The server receives the requested filename from the client.
- It checks whether the file exists.
- If the file is too small (< 2000 bytes), an error message is sent.
- If the file exists, it is opened and divided into **4 segments**.

### 4. Fragmentation & Checksum Calculation:

- Each segment is assigned a **sequence number**.
- A **16-bit checksum** is calculated using a one's complement method.
- The segment and checksum are packed into a structured message.

## 5. Sending File Segments:

- Each segment is sent to the client.
- The server waits for an **ACK** response.
- If a **NACK** is received (indicating corruption), the segment is retransmitted (max 5 retries).
- If retries are exhausted, the server aborts the transmission.

## 5. Problems Encountered & Solutions

| Challenge                    | Solution                                         |
|------------------------------|--------------------------------------------------|
| Handling packet loss         | Implemented retransmissions with max retries     |
| Data integrity verification  | Used a 16-bit checksum validation                |
| Fragmentation and reordering | Implemented sequence numbers for proper ordering |
| Multi-client handling        | Used threading to manage multiple connections    |

## 6. Screenshots of the given Output

```
PS C:\Users\airfi\OneDrive\Desktop\IMAMU\LVL7\c330\Project\Server> python TCP_Server.py
Server is listening on 0.0.0.0:8473
New client connected from ('192.168.0.105', 18441)
Received request for file: example.txt
Starting file transfer for: example.txt
[SERVER] Sending segment 1/4 with size 500 and checksum 22526
[SERVER] Received response for segment 1: ACK
[SERVER] Segment 1 acknowledged.
[SERVER] Sending segment 2/4 with size 500 and checksum 41599
[SERVER] Received response for segment 2: ACK
[SERVER] Segment 2 acknowledged.
[SERVER] Sending segment 3/4 with size 500 and checksum 16022
[SERVER] Received response for segment 3: ACK
[SERVER] Segment 3 acknowledged.
[SERVER] Sending segment 4/4 with size 500 and checksum 37791
[SERVER] Received response for segment 4: ACK
[SERVER] Segment 4 acknowledged.
[SERVER] End-of-transmission signal sent.
[SERVER] File transfer completed successfully.
Client disconnected.
Exception in thread Thread-1 (handle_client):
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.13.3.13.496.0_x64__qbz5n2kfra8p8\Lib\threading.py", line 1041, in _bootstrap_inner
    self.run()
    ~~~~~^~~~~
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.13.3.13.496.0_x64__qbz5n2kfra8p8\Lib\threading.py", line 992, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~^~~~~~
  File "C:\Users\airfi\OneDrive\Desktop\IMAMU\LVL7\c330\Project\Server\TCP_Server.py", line 98, in handle_client
    file_name = conn.recv(1024).decode().strip()
    ~~~~~^~~~~~
TimeoutError: timed out
|
```

```
PS C:\Users\hmb16\Desktop> python TCP_Client.py
Enter Server IP: 192.168.0.107
Connected to server 0.0.0.0:8473
Enter File Name: example.txt
Server response: Start transfer
[CLIENT] Waiting to receive segment...
[CLIENT] Sequence Number: 1
[CLIENT] Received Checksum: 22526
[CLIENT] Segment Size: 500
[CLIENT] Received Segment 1 with size 500 and checksum 22526
[CLIENT] Fully read segment 1 with size 500
[CLIENT] Calculated checksum for segment 1: 22526
[CLIENT] Segment 1 received correctly, sending ACK.
[CLIENT] Waiting to receive segment...
[CLIENT] Sequence Number: 2
[CLIENT] Received Checksum: 41599
[CLIENT] Segment Size: 500
[CLIENT] Received Segment 2 with size 500 and checksum 41599
[CLIENT] Fully read segment 2 with size 500
[CLIENT] Calculated checksum for segment 2: 41599
[CLIENT] Segment 2 received correctly, sending ACK.
[CLIENT] Waiting to receive segment...
[CLIENT] Sequence Number: 3
[CLIENT] Received Checksum: 16022
[CLIENT] Segment Size: 500
[CLIENT] Received Segment 3 with size 500 and checksum 16022
[CLIENT] Fully read segment 3 with size 500
[CLIENT] Calculated checksum for segment 3: 16022
[CLIENT] Segment 3 received correctly, sending ACK.
[CLIENT] Waiting to receive segment...
[CLIENT] Sequence Number: 4
[CLIENT] Received Checksum: 37791
[CLIENT] Segment Size: 500
[CLIENT] Received Segment 4 with size 500 and checksum 37791
[CLIENT] Fully read segment 4 with size 500
[CLIENT] Calculated checksum for segment 4: 37791
[CLIENT] Segment 4 received correctly, sending ACK.
[CLIENT] Waiting to receive segment...
[CLIENT] Sequence Number: 1128809795
[CLIENT] Received Checksum: 19295
[CLIENT] Segment Size: 1129270862
[CLIENT] ERROR: Connection closed while receiving segment.
[CLIENT] File received successfully.
Enter File Name: |
```